Sequences and Structure

Week 6 — Deep Learning II & Unsupervised Learning (ISLP Ch. 10.5–10.8, Ch. 12)

Ho-min Park

2026-08-14

Welcome Back

Where we are

- **Last week** — SVMs, then neural networks: layers, activations, CNNs for images
- **Today, part 1** — networks for **sequences** (text, time series), and how any network is actually *fitted* (SGD, backprop, dropout)
- **Today, part 2** — a bigger shift: **what can we learn when there is no Y at all?** PCA and clustering

Everything until today has been supervised. Unsupervised learning drops the answer key — which changes what "being right" even means.

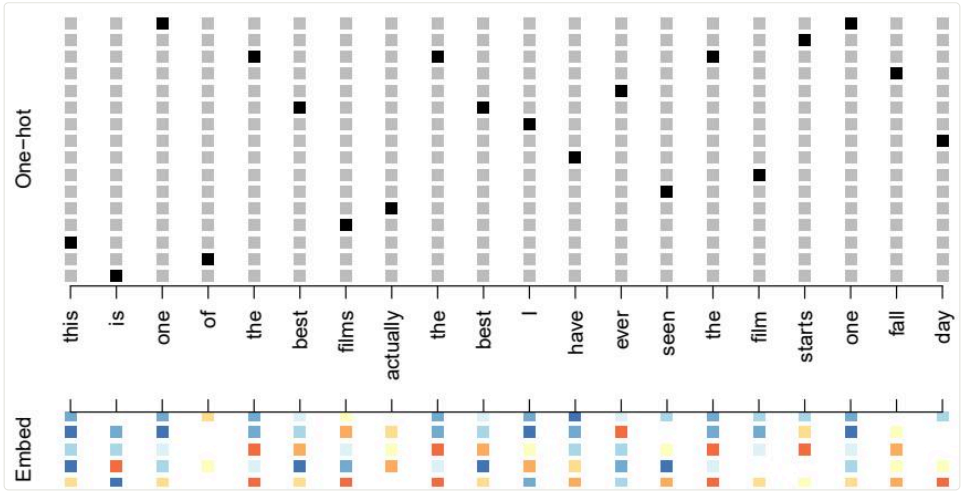
Deep Learning II: Sequences

When the input has an order

A document, a stock-price history, a genome, a sensor log — each is a sequence $X = \{X_1, X_2, \ldots, X_L\}$ where **the order carries the information**.

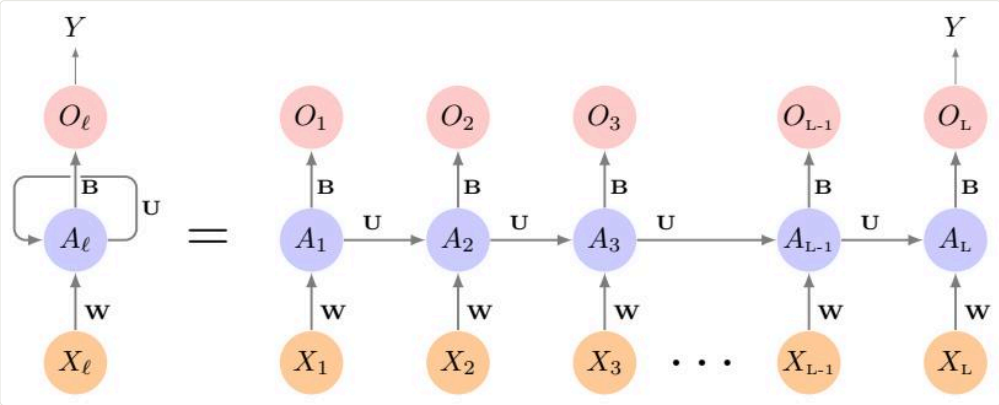
- Bag-of-words models throw the order away — “good, not bad” and “bad, not good” become identical
- Fixed-input methods (everything we’ve seen) need a new idea for variable-length, ordered data

First problem: words aren't numbers



One-hot encoding (top): each word = a huge sparse indicator vector — every pair of words equally far apart. An **embedding** (bottom) maps each word to a short dense vector where **similar words land close together** — learned from data, or borrowed (word2vec, GloVe).

The recurrent idea: reuse one network, carry a memory

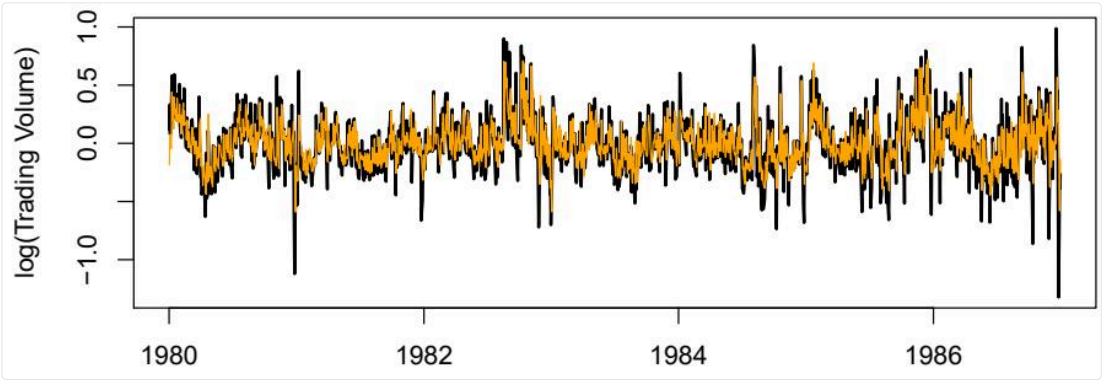


Process the sequence one element at a time; the hidden **activation** A_ℓ feeds forward into the next step. Crucially, the weights $\mathbf{W}, \mathbf{U}, \mathbf{B}$ are **the same at every step** — one network, unrolled L times, with a running summary as memory.

RNNs in practice

- Plain RNNs forget quickly — long sequences drown the early signal
- **LSTM** and **GRU** cells add gates that decide what to keep and what to forget; the workhorses of pre-transformer NLP
- Sequence-to-sequence RNNs (translate, summarize) are the **direct ancestors of today's transformer LLMs** — same problem, new architecture

Time series: predicting NYSE trading volume



An RNN fed the last 5 days of (volume, return, volatility) predicts tomorrow's log volume — capturing **42% of test-set variance**. Sobering footnote: a plain autoregressive linear model on lagged values does **almost exactly as well**.

When to actually use deep learning

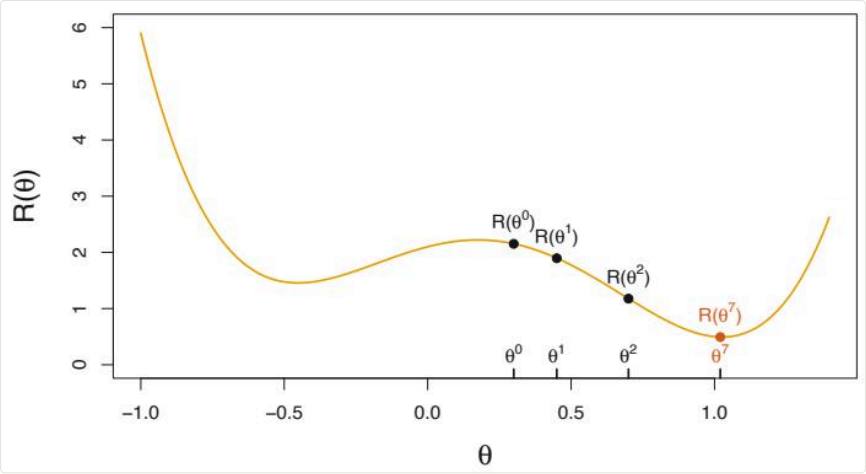
ISLP's own advice (§10.6), after fitting all these networks:

- On `Hitters`, a lasso model matches the neural network — with 12 interpretable coefficients
- Deep learning shines with **large n, and signal-rich raw inputs** (images, audio, text) — not on every tabular problem
- **Occam's razor**: given similar performance, take the simpler, more interpretable model

The best argument for learning Weeks 2–4 thoroughly: they're the baseline deep learning has to beat — and often doesn't.

How Networks Are Fitted

A non-convex problem



$R(\theta) = \frac{1}{2}\sum_i (y_i - f_\theta(x_i))^2$ has **many minima** — unlike least squares, no formula, no unique answer.

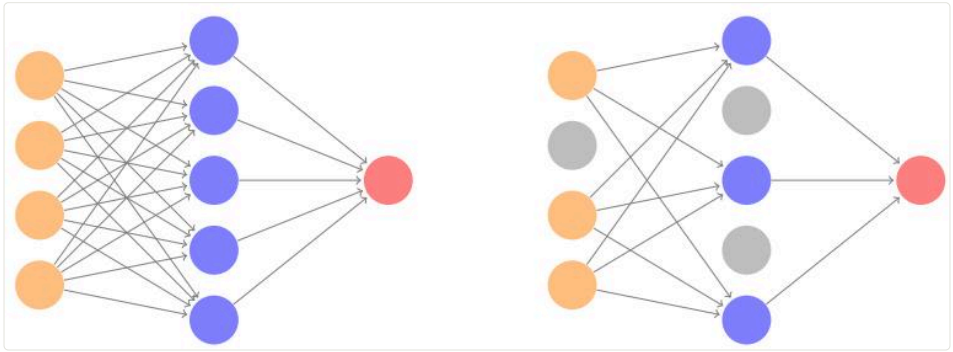
Gradient descent: start somewhere, repeatedly step downhill: $\theta^{t+1} = \theta^t - \rho \nabla R(\theta^t)$, with learning rate ρ.

Backpropagation and SGD

- **Backpropagation** = the chain rule, organized: the gradient of the loss flows backwards through the layers, one cheap pass
- **Stochastic** gradient descent: compute each step's gradient on a small **minibatch**, not all n points — noisier steps, far cheaper, and the noise even helps escape bad minima
- One sweep through the data = an **epoch**

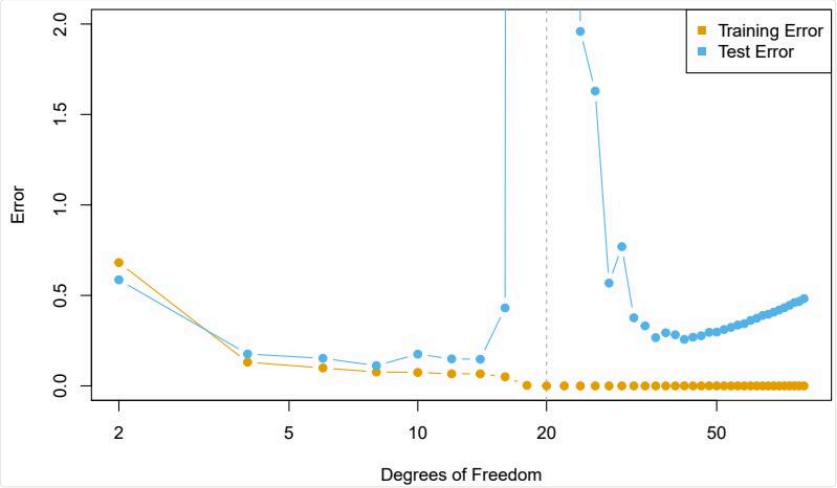
None of this guarantees the global minimum — and in practice, **nobody cares**: a good local minimum generalizes fine.

Keeping big networks honest: dropout



Networks have *far* more parameters than observations — regularization is not optional. Options: ridge penalties on weights, **early stopping**, and **dropout** — randomly silence a fraction of units at each training step, so no unit can free-ride on another. **The ensemble idea from Week 4**, happening inside one network.

Double descent: a plot twist



Push flexibility *past* the point of interpolating the training data (zero training error) and test error can come **back down**. Doesn't break the bias-variance trade-off — among all interpolating fits, the training procedure finds the *smoothest* one. But it explains why absurdly overparameterized networks can still generalize.

This week's lab, part 1: an RNN in PyTorch

```
import torch.nn as nn

class NYSEModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(3, 12, batch_first=True)
        self.dense = nn.Linear(12, 1)
    def forward(self, x):
        val, h_n = self.rnn(x)
        return self.dense(val[:, -1])   # last time step only
```

Lab 10.9.5–10.9.6: IMDB sentiment with embeddings, then this NYSE forecaster — plus the autoregressive baseline that keeps it humble.

Unsupervised Learning

No Y, new rules

Only features $X_1, \ldots, X_p$ — no response to predict. The goal shifts to **discovery**: interesting directions, informative groups.

- Is there a low-dimensional way to *see* this data? → **PCA**
- Do the observations fall into *groups*? → **clustering**

The honest difficulty: **no test error exists**. There's no ground truth to check against — results must be judged by usefulness, stability, and domain sense. More art, more caution.

PCA: the directions that matter most

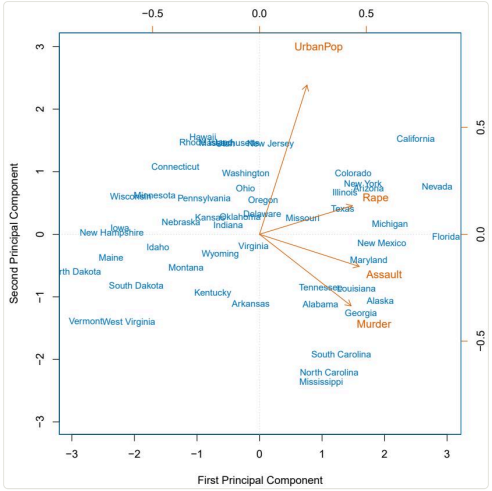
The **first principal component** is the normalized linear combination

$$Z_1 = \phi_{11} X_1 + \phi_{21} X_2 + \cdots + \phi_{p1} X_p$$

with the largest variance — the single axis along which the data varies most. Z_2 is the next-most-variable direction *uncorrelated with* Z_1, and so on.

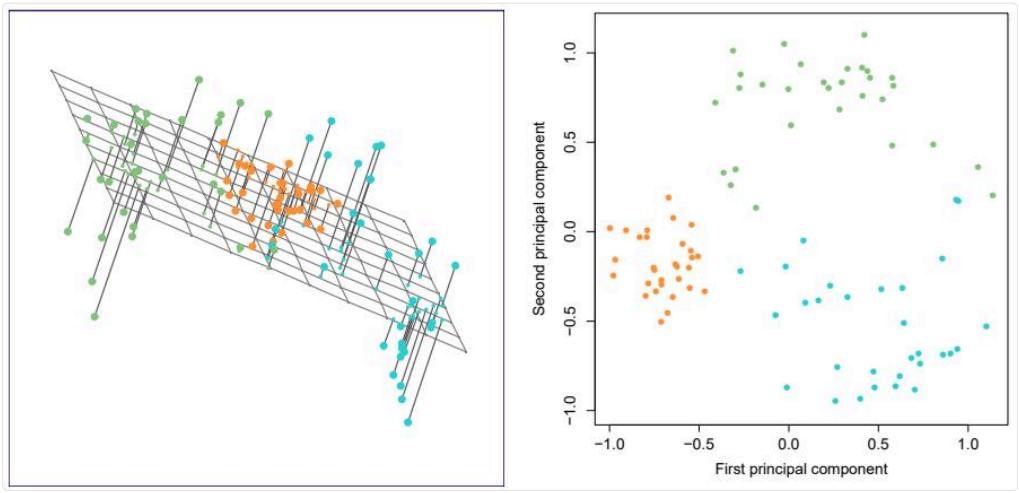
- The ϕ's are **loadings** (what the direction is made of)
- The z_i's are **scores** (where each observation sits on it)

Fifty states, four crimes, one picture



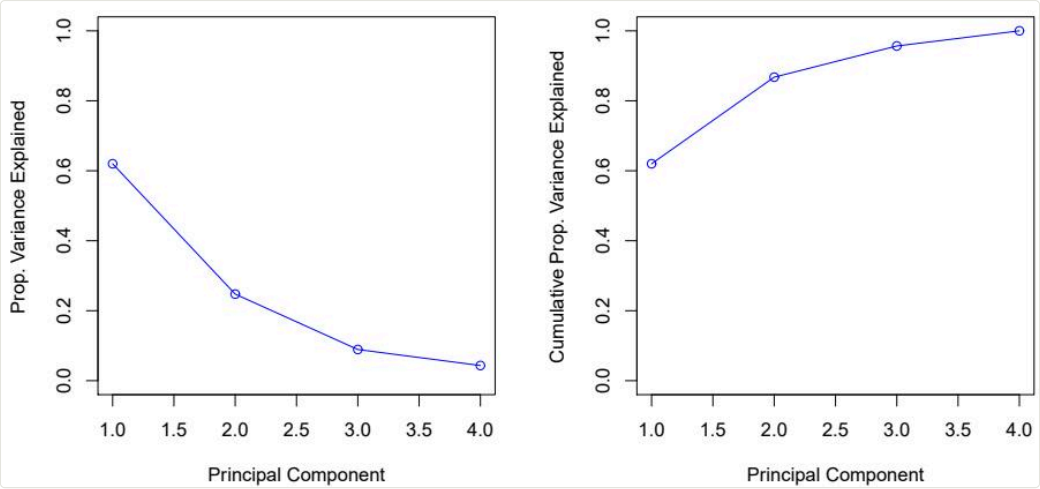
`USArrests`: PC1 loads on the three crime rates (overall crime level), PC2 on urbanization. The **biplot** shows both scores (states) and loadings (arrows): California high on both; the Dakotas low on both. **Four dimensions compressed into a readable map.**

Another view: the best-fitting flat surface



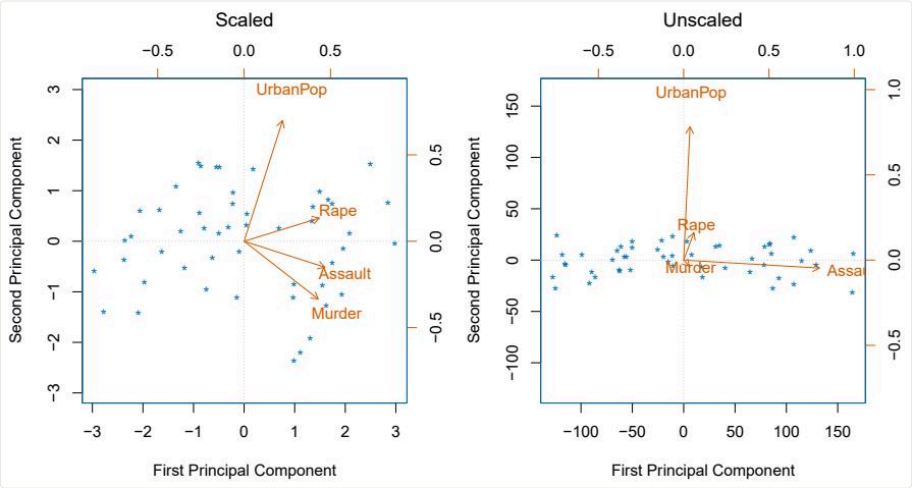
Equivalent definition: the first M components span the flat surface **closest to the data points** (minimal total squared distance). PCA = the best possible M-dimensional shadow of a p-dimensional cloud. (This view powers matrix completion — recommender systems impute missing entries with iterated PCA, §12.3.)

How many components? The scree plot



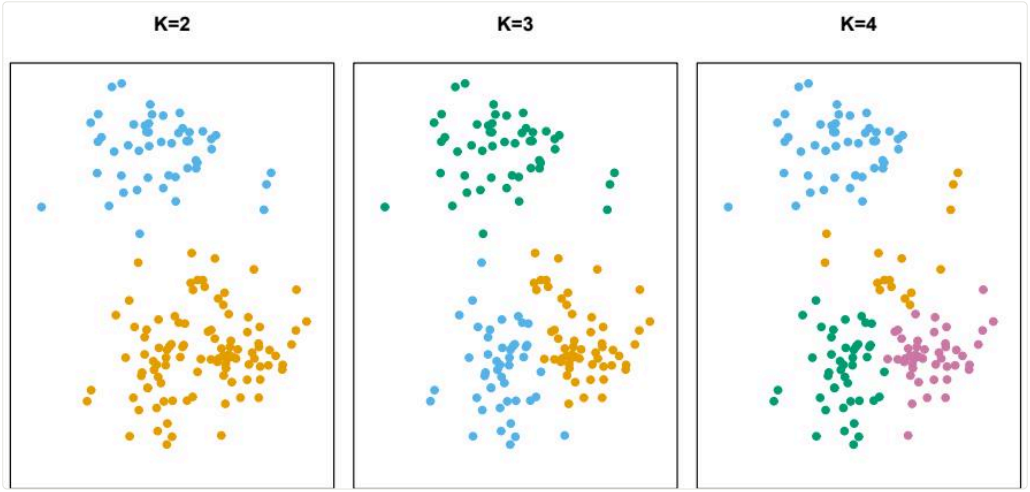
Each PC explains a **proportion of variance** (PVE). For `USArrests`: 62%, 25%, 9%, 4% — two components carry 87%. The standard heuristic: look for the **elbow** where the plot flattens. Unsatisfying but honest — **there is no CV to hide behind here**.

Scale first, or one variable runs the show



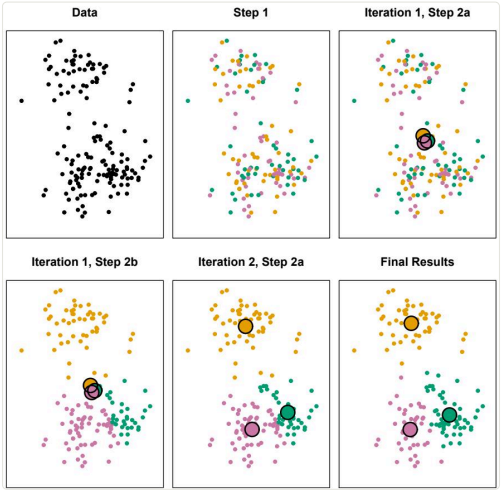
Unscaled (right), `Assault` — the variable with the biggest raw variance — hijacks PC1 purely because of its units. **Standardize each variable (mean 0, sd 1) before PCA** unless everything is measured in the same units on purpose.

K-means: pick K, minimize within-cluster scatter



Partition the observations into K clusters minimizing total **within-cluster variation** $\sum_{k=1}^{K} W(C_k)$ (squared Euclidean distances). You choose K — and each choice tells **a different, equally confident-looking story**.

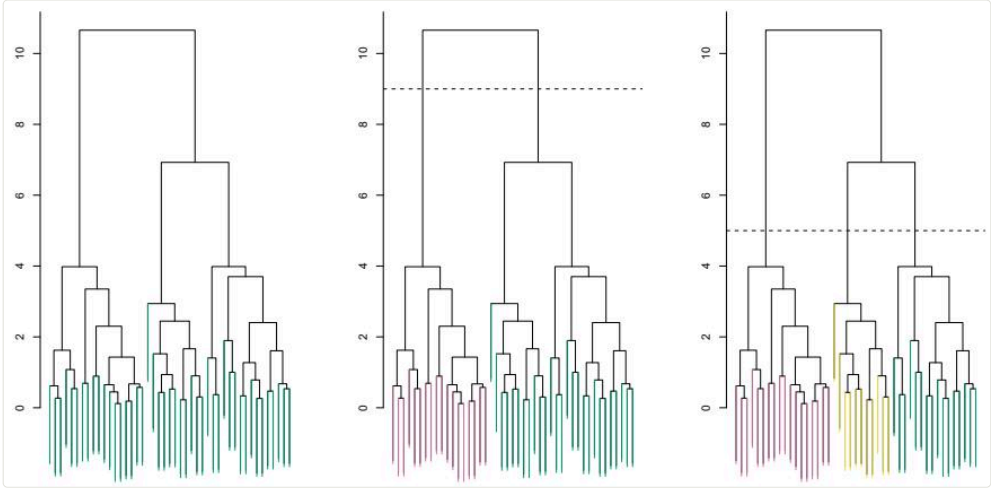
The algorithm: assign, average, repeat



1. Random initial assignment → 2. compute cluster **centroids** →
2. reassign each point to its nearest centroid → repeat until nothing moves.

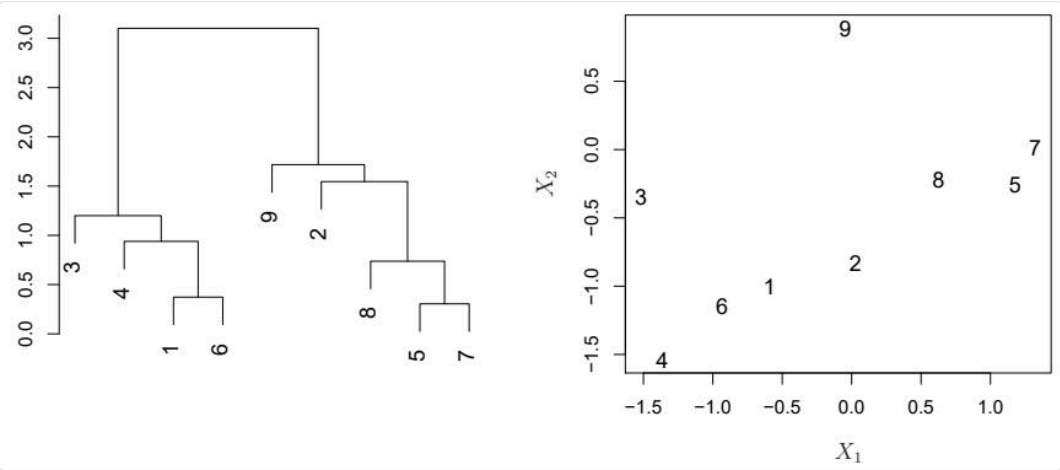
Each step lowers the objective, but only to a **local optimum** — run many random starts (`n_init` in sklearn) and keep the best.

Hierarchical clustering: don’t choose K, grow a tree



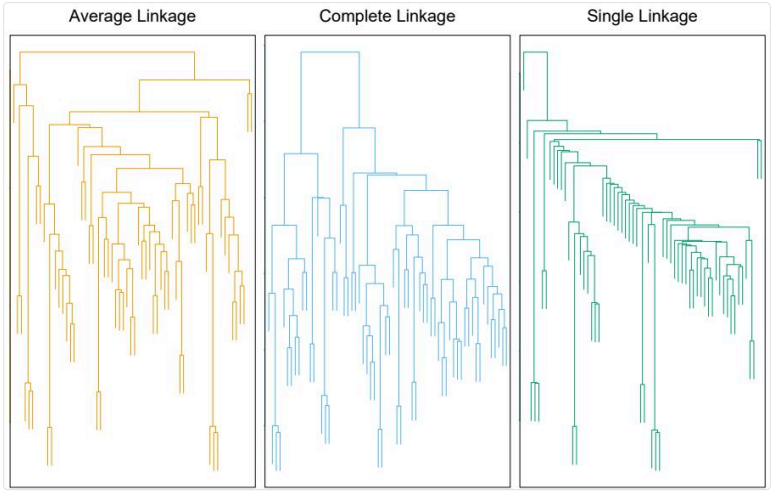
Start with n singleton clusters; repeatedly **fuse the two closest** clusters until one remains. The record of fusions is a **dendrogram** — cut it at height 9 for two clusters, at 5 for three. **One tree, every K at once.**

Reading a dendrogram (most people do it wrong)



Similarity is the **height where two observations first fuse** — never their left-right closeness. Here observation 9 looks adjacent to 2, but is no more similar to 2 than to 8, 5, or 7: they all join 9 at the same height. Horizontal position is **an artifact of drawing**.

The knobs: linkage and dissimilarity



- **Linkage** = distance between *clusters*: complete (max), average, single (min — produces stringy, unbalanced trees). Average/complete preferred
- **Dissimilarity**: Euclidean distance, or *correlation-based* (shapes of profiles, not magnitudes) — the right choice is a **domain decision**, e.g. shopper baskets vs gene expression profiles

Unsupervised results: handle with care

- Scaling, dissimilarity, linkage, K — **small decisions, big consequences**; try several, report what's stable
- Clusters *will* appear even in noise — the algorithms have no way to say "there's nothing here"
- Validate: do the clusters persist across subsamples? Do they mean something to a domain expert?
- Use as the start of a hypothesis, not the end of an analysis

This week's lab, part 2: three lines each

```
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import dendrogram, linkage

scores = PCA().fit_transform(X_scaled)
km = KMeans(n_clusters=3, n_init=20).fit(X_scaled)
dendrogram(linkage(X_scaled, method='complete'))
```

Lab 12.5: PCA on `USArrests`, matrix completion, K-means and hierarchical clustering — ending with the `NCI60` cancer cell lines, where clustering meets real genomics.

Getting Started

Before next Friday

1. Read **ISLP Ch. 11 and Ch. 13** (`week7/ch11-survival-analysis.pdf`, `week7/ch13-multiple-testing.pdf`) — Survival Analysis and Multiple Testing, for the final session on Aug 21
2. Run this week's labs: **10.9.5–10.9.6** (IMDB & RNNs) and **12.5** (PCA & clustering)
3. Try it on your own data: PCA-then-cluster something from your work — and notice how much the story changes with scaling on vs off
4. Final week wraps up with two topics every practitioner eventually needs: time-to-event data and the perils of testing many hypotheses

Questions?

See you next Friday — one week to go.